

Assignment No:2

Q.1 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling.

the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process.

Also find the average waiting time and turnaround time.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
struct Process
{
    int pid;
    int arrivalTime;
    int burstTime;
    int completionTime;
    int turnaroundTime;
    int waitingTime;
    int completed;
};
int main()
{
    int n, i, j;
    int currentTime = 0;
    int completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n]
    for (i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("\nEnter Arrival Time for P%d: ", i + 1);
        scanf("%d", &p[i].arrivalTime);
        printf("Enter First CPU Burst for P%d: ", i + 1);
```

```

        scanf("%d", &p[i].burstTime);
        p[i].completed = 0;
    }
    srand(time(NULL));
    printf("\nRandomly generated CPU bursts:\n");
    for (i = 0; i < n; i++)
    {
        int randomBurst = (rand() % 10) + 1;
        printf("P%d : First Burst = %d, Next Burst = %d\n",
            p[i].pid, p[i].burstTime, randomBurst);
        p[i].burstTime = randomBurst;
    }
    printf("\n\nGANTT CHART\n");
    printf("-----\n");
    while (completed < n)
    {
        int shortest = -1;
        for (i = 0; i < n; i++)
        {
            if (p[i].completed == 0 &&
                p[i].arrivalTime <= currentTime)
            {
                if (shortest == -1 ||
                    p[i].burstTime < p[shortest].burstTime)
                {
                    shortest = i;
                }
            }
        }

        if (shortest == -1)
        {
            currentTime++;
            printf("| IDLE ");

```

```

        continue;
    }
    printf("| P%d ", p[shortest].pid);
    currentTime = currentTime + p[shortest].burstTime;
    p[shortest].completionTime = currentTime
    p[shortest].turnaroundTime =
        p[shortest].completionTime - p[shortest].arrivalTime;
    p[shortest].waitingTime =
        p[shortest].turnaroundTime - p[shortest].burstTime;
    p[shortest].completed = 1;
    completed++;
}
printf("\n");
printf("-----\n");
printf("\nPROCESS DETAILS\n");
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
float totalWaitingTime = 0;
float totalTurnaroundTime = 0;
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrivalTime,
        p[i].burstTime,
        p[i].completionTime,
        p[i].turnaroundTime,
        p[i].waitingTime);
    totalWaitingTime += p[i].waitingTime;
    totalTurnaroundTime += p[i].turnaroundTime;
}
printf("\nAverage Waiting Time = %.2f",
    totalWaitingTime / n);
printf("\nAverage Turnaround Time = %.2f\n",

```

```

        totalTurnaroundTime / n);
    return 0;
}

```

Q.2 Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
struct Process
{
    int pid;
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int completionTime;
    int turnaroundTime;
    int waitingTime;
};
int main()
{
    int n, quantum;
    int i;
    int currentTime = 0;
    int completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (i = 0; i < n; i++)
    {

```

```

    p[i].pid = i + 1;
    printf("\nEnter Arrival Time for P%d: ", i + 1);
    scanf("%d", &p[i].arrivalTime);
    printf("Enter First CPU Burst for P%d: ", i + 1);
    scanf("%d", &p[i].burstTime);
}
printf("\nEnter Time Quantum: ");
scanf("%d", &quantum);
srand(time(NULL));
printf("\nRandomly generated next CPU bursts:\n");
for (i = 0; i < n; i++)
{
    int randomBurst = (rand() % 10) + 1;
    printf("P%d : First Burst = %d, Next Burst = %d\n",
        p[i].pid,
        p[i].burstTime,
        randomBurst);
    p[i].burstTime = randomBurst;
    p[i].remainingTime = p[i].burstTime;
    p[i].completionTime = 0;
    p[i].turnaroundTime = 0;
    p[i].waitingTime = 0;
}
printf("\n\nGANTT CHART\n");
printf("-----\n");
while (completed < n)
{
    int processExecuted = 0;
    for (i = 0; i < n; i++)
    {
        if (p[i].arrivalTime <= currentTime &&
            p[i].remainingTime > 0)
        {
            processExecuted = 1;

```

```

printf("| P%d ", p[i].pid);
if (p[i].remainingTime > quantum)
{
    currentTime += quantum;
    p[i].remainingTime -= quantum;
}
else
{
    currentTime += p[i].remainingTime;
    p[i].remainingTime = 0;
    p[i].completionTime = currentTime;
    p[i].turnaroundTime =
        p[i].completionTime -
        p[i].arrivalTime;
    p[i].waitingTime =
        p[i].turnaroundTime -
        p[i].burstTime
    completed++;
}
}
}

if (processExecuted == 0)
{
    currentTime++;
    printf("| IDLE ");
}
}

printf("\n");
printf("-----\n");
printf("\nPROCESS DETAILS\n");
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
float totalWaitingTime = 0;

```

```

float totalTurnaroundTime = 0;
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrivalTime,
        p[i].burstTime,
        p[i].completionTime,
        p[i].turnaroundTime,
        p[i].waitingTime);
    totalWaitingTime += p[i].waitingTime;
    totalTurnaroundTime += p[i].turnaroundTime;
}
printf("\nAverage Waiting Time = %.2f",
    totalWaitingTime / n);
printf("\nAverage Turnaround Time = %.2f\n",
    totalTurnaroundTime / n);
return 0;
}

```

Q.3 Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>

#include <stdlib.h>
#include <time.h>

struct Process
{
    int pid;
    int arrivalTime;

```

```

    int burstTime;
    int completionTime;
    int turnaroundTime;
    int waitingTime;
};

int main()
{
    int n, i;
    int currentTime = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("\nEnter Arrival Time for P%d: ", i + 1);
        scanf("%d", &p[i].arrivalTime);
        printf("Enter First CPU Burst for P%d: ", i + 1);
        scanf("%d", &p[i].burstTime);
    }
    srand(time(NULL));
    printf("\nRandomly generated next CPU bursts:\n");
    for (i = 0; i < n; i++)
    {
        int randomBurst = (rand() % 10) + 1;
        printf("P%d : First Burst = %d, Next Burst = %d\n",
            p[i].pid,
            p[i].burstTime,
            randomBurst);
        p[i].burstTime = randomBurst;
    }
    for (i = 0; i < n - 1; i++)
    {
        int j;

```

```

for (j = 0; j < n - i - 1; j++)
{
    if (p[j].arrivalTime > p[j + 1].arrivalTime)
    {
        struct Process temp = p[j];
        p[j] = p[j + 1];
        p[j + 1] = temp;
    }
}
}
printf("\n\nGANTT CHART\n");
printf("-----\n");
for (i = 0; i < n; i++)
{
    if (currentTime < p[i].arrivalTime)
    {
        printf("| IDLE ");
        currentTime = p[i].arrivalTime;
    }
    printf("| P%d ", p[i].pid);
    currentTime = currentTime + p[i].burstTime;
    p[i].completionTime = currentTime;
    p[i].turnaroundTime = p[i].completionTime - p[i].arrivalTime;
    p[i].waitingTime = p[i].turnaroundTime - p[i].burstTime;
}
printf("\n");
printf("-----\n");
printf("\nPROCESS DETAILS\n");
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
float totalWaitingTime = 0;
float totalTurnaroundTime = 0;
for (i = 0; i < n; i++)
{

```

```

printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    p[i].pid,
    p[i].arrivalTime,
    p[i].burstTime,
    p[i].completionTime,
    p[i].turnaroundTime,
    p[i].waitingTime);
totalWaitingTime += p[i].waitingTime;
totalTurnaroundTime += p[i].turnaroundTime;
}
printf("\nAverage Waiting Time = %.2f",
    totalWaitingTime / n);
printf("\nAverage Turnaround Time = %.2f\n",
    totalTurnaroundTime / n);
return 0; }

```

Q.4 Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time .

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
struct Process
{
    int pid;
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int priority;
    int completionTime;
    int turnaroundTime;
    int waitingTime;
};

```

```

int main()
{
    int n, i;
    int currentTime = 0;
    int completed = 0;
    int lastProcess = -1;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("\nEnter Arrival Time for P%d: ", i + 1);
        scanf("%d", &p[i].arrivalTime);
        printf("Enter First CPU Burst for P%d: ", i + 1);
        scanf("%d", &p[i].burstTime);
        printf("Enter Priority for P%d: ", i + 1);
        scanf("%d", &p[i].priority);
    }
    srand(time(NULL));
    printf("\nRandomly generated next CPU bursts:\n");
    for (i = 0; i < n; i++)
    {
        int randomBurst = (rand() % 10) + 1;
        printf("P%d : First Burst = %d, Next Burst = %d\n",
            p[i].pid,
            p[i].burstTime,
            randomBurst);
        p[i].burstTime = randomBurst;
        p[i].remainingTime = p[i].burstTime;
        p[i].completionTime = 0;
        p[i].turnaroundTime = 0;
        p[i].waitingTime = 0;
    }
}

```

```

printf("\n\nGANTT CHART\n");
printf("-----\n");
while (completed < n)
{
    int highestPriority = -1;
    for (i = 0; i < n; i++)
    {
        if (p[i].arrivalTime <= currentTime &&
            p[i].remainingTime > 0)
        {
            if (highestPriority == -1 ||
                p[i].priority < p[highestPriority].priority)
            {
                highestPriority = i;
            }
        }
    }
    if (highestPriority == -1)
    {
        if (lastProcess != -2)
        {
            printf("| IDLE ");
            lastProcess = -2;
        }
        currentTime++;
        continue;
    }
    if (lastProcess != highestPriority)
    {
        printf("| P%d ", p[highestPriority].pid);
        lastProcess = highestPriority;
    }
    p[highestPriority].remainingTime--;
    currentTime++;
}

```

```

if (p[highestPriority].remainingTime == 0)
{
    p[highestPriority].completionTime = currentTime;
    p[highestPriority].turnaroundTime = p[highestPriority].completionTime -
        p[highestPriority].arrivalTime;
    p[highestPriority].waitingTime = p[highestPriority].turnaroundTime -
        p[highestPriority].burstTime;
    completed++;
}
}
printf("\n");
printf("-----\n");
printf("\nPROCESS DETAILS\n");
printf("\nPID\tAT\tBT\tP\tCT\tTAT\tWT\n");
printf("-----\n");
float totalWaitingTime = 0;
float totalTurnaroundTime = 0;
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrivalTime,
        p[i].burstTime,
        p[i].priority,
        p[i].completionTime,
        p[i].turnaroundTime,
        p[i].waitingTime);
    totalWaitingTime += p[i].waitingTime;
    totalTurnaroundTime += p[i].turnaroundTime;
}
printf("\nAverage Waiting Time = %.2f",
    totalWaitingTime / n);
printf("\nAverage Turnaround Time = %.2f\n",
    totalTurnaroundTime / n);

```

```
    return 0;
}
```

Q.5 Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
struct Process
{
    int pid;
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int completionTime;
    int turnaroundTime;
    int waitingTime;
};
int main()
{
    int n, i;
    int currentTime = 0;
    int completed = 0;
    int lastProcess = -1;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
```

```

for (i = 0; i < n; i++)
{
    p[i].pid = i + 1;
    printf("\nEnter Arrival Time for P%d: ", i + 1);
    scanf("%d", &p[i].arrivalTime);
    printf("Enter First CPU Burst for P%d: ", i + 1);
    scanf("%d", &p[i].burstTime);
}

srand(time(NULL));
printf("\nRandomly generated next CPU bursts:\n");
for (i = 0; i < n; i++)
{
    int randomBurst = (rand() % 10) + 1;
    printf("P%d : First Burst = %d, Next Burst = %d\n",
        p[i].pid,
        p[i].burstTime,
        randomBurst);
    p[i].burstTime = randomBurst;
    p[i].remainingTime = p[i].burstTime;
    p[i].completionTime = 0;
    p[i].turnaroundTime = 0;
    p[i].waitingTime = 0;
}

printf("\n\nGANTT CHART\n");
printf("-----\n");
while (completed < n)
{
    int shortest = -1;
    for (i = 0; i < n; i++)
    {
        if (p[i].arrivalTime <= currentTime &&
            p[i].remainingTime > 0)
        {
            if (shortest == -1 ||

```

```

        p[i].remainingTime < p[shortest].remainingTime)
    {
        shortest = i;
    }
}
}
if (shortest == -1)
{
    if (lastProcess != -2)
    {
        printf("| IDLE ");
        lastProcess = -2;
    }
    currentTime++;
    continue;
}
if (lastProcess != shortest)
{
    printf("| P%d ", p[shortest].pid);
    lastProcess = shortest;
}
p[shortest].remainingTime--;
currentTime++;
if (p[shortest].remainingTime == 0)
{
    p[shortest].completionTime = currentTime;
    p[shortest].turnaroundTime =
        p[shortest].completionTime -
        p[shortest].arrivalTime;
    p[shortest].waitingTime =
        p[shortest].turnaroundTime -
        p[shortest].burstTime;
    completed++;
}

```

```

}
printf("\n");
printf("-----\n");
printf("\nPROCESS DETAILS\n");
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");
float totalWaitingTime = 0;
float totalTurnaroundTime = 0;
for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].arrivalTime,
        p[i].burstTime,
        p[i].completionTime,
        p[i].turnaroundTime,
        p[i].waitingTime);
    totalWaitingTime += p[i].waitingTime;
    totalTurnaroundTime += p[i].turnaroundTime;
}
printf("\nAverage Waiting Time = %.2f",
    totalWaitingTime / n);
printf("\nAverage Turnaround Time = %.2f\n",
    totalTurnaroundTime / n);
return 0;
}

```

